AfterFix: Evaluating Data Recovery with Supplied SQL Patches

A controlled study of adapting supplied SQL patches to repair existing data

Nitish Songara
Independent researcher

Abstract

A correct code patch can prevent new errors while leaving existing data wrong. AfterFix tests this recovery task on 24 synthetic SQLite cases. Agents receive a patch validated on the evaluated fixtures, complete evidence and permission to rebuild derived tables. We report three separate evaluations: the original 72-attempt Gemma 4 12B study, a three-attempt correction for a testing-system bug, and a 144-attempt follow-up comparing 12B and 31B. In the follow-up, ordinary tools achieve full recovery on 11/24 cases with 12B and 18/24 with 31B. Checklist prompts and optional dependency tracing show no consistent benefit. A later CPU check shows that removing filters from the supplied patches solves all 24 cases, limiting the task to adapting existing logic. All 132 follow-up repairs that execute successfully leave a stable state when repeated, including 48 incorrect repairs. These findings describe a small controlled study with limited difficulty and production relevance.

1. Introduction and contribution

A patch may correctly process newly changed records while skipping older records damaged before the fix. Recovery must repair that existing data and preserve legitimate changes made since the incident. This problem is established: WARP studies recovery with retroactive patches [1], and QFix studies repair through query histories [2].

Our question is narrower: given application SQL validated on the evaluated fixtures and explicit recovery requirements, can an agent adapt the SQL to repair existing data across several versions of a test case? We compare ordinary tools, checklist prompts and optional dependency tracing. We check whether execution and stability identify correct repairs.

Our contribution is an inspectable task set and controlled evaluation of supplied-patch adaptation.

For data engineers, the intended value is explicit evidence authority, permitted changes and recovery criteria. We have not measured production benefits or established a new recovery algorithm or first benchmark.

2. Tasks and test data

Each case provides business rules, schema, records, immutable evidence, declared dependencies and application SQL validated on its recorded fixture transitions. The writable tables are **projections**: stored results derived from the evidence. The host runs the supplied application patch unchanged; the agent generates separate recovery SQL.

The patch updates records listed in a table called `work`, which represents the current batch of changed records. Historical test cases have an empty `work` table. All 24 cases allow full rebuilding of the projections. None requires repairs that only append new records. These choices provide the agent with both the reconstruction logic and permission to use it broadly.

There are four cases in each of six groups: delivery workflows, coupled resources, versioned storage, authorization graphs, ordered state and evidence adjudication. All cases are authored examples. We create damaged states by modifying correct projections, without executing an original faulty application.

Agents can query the initial database, test recovery SQL on a fresh copy, and submit a repair. Public test feedback includes execution errors, resulting projection rows, write counts and whether repetition changes the state. It does not reveal expected answers or private correctness scores. Changes from one test do not carry into the next. We do not evaluate minimal writes, uninterrupted service or effects outside the database.

Python **oracles** calculate expected results without executing reference SQL. Before model evaluation, two reference strategies per case passed all three variants, and seven incorrect executable strategies per case failed: 144 positive and 504 negative checks. Cases and oracles were developed within the same AI-assisted project; separate implementations are not independent human review.

Seeds 0, 7 and 19 change identifiers and selected numeric values while keeping each case's structure. We run the same generated repair on all three variants. This detects repairs tied to example IDs or values; it does not test new schemas or incident types. Development cases were excluded from the original evaluation, but the development process had access to test cases. Local file hashes record the frozen design and help detect changes; they are not an external record of advance registration.

3. Checks, tools and experiment settings

Full recovery means passing all six checks on all three variants:

Check	What must hold
Historical state	Repaired projections match the oracle's expected results
Preservation	Trusted evidence is unchanged, and previously correct projection rows remain present
Repeat-state stability	Running recovery again leaves the database state unchanged
Healthy control	Recovery leaves an already-correct database unchanged
Future patch	The supplied patch processes new evidence correctly and repeatably on a healthy database
Continuation	After recovery, the supplied patch processes new evidence correctly and repeatably

Preservation checks the final state, so a permitted delete-and-rebuild repair can pass. Stable repetition need not mean zero repeated writes. The future-patch check validates supplied code, not code written by the model. Execution and stability counts require passing their checks on all three variants. We distinguish execution failures, missing repairs and repairs that execute but are wrong. **Visible recovery** is the final repair's full score on seed 0, the variant shown to the agent.

All three methods receive the same records, evidence and dependency edges. Edges declare dependencies between tables or concepts. **Ordinary tools** already instruct the agent to recover data, preserve valid state and check repetition. **Checklist** adds organized reminders of these requirements. **Trace access** adds a tool for following the declared edges. It returns existing nodes, edges and evidence references, without

private information, damaged-row identification or repair planning. It is not a record-level history of how data was produced.

The agent returns one JSON action per turn. Queries, tests, trace calls, submissions and malformed responses each consume a turn. There are no free formatting retries. A submission ends the attempt. If a limit is reached, we grade the latest test or submission that had the required JSON fields, even if its SQL fails. An attempt without a valid repair fails. Case and method order is balanced within each model.

Setting	**Study A: original**	**Study B: v3 follow-up**
Model / weight format	Gemma 4 12B / prepared NF4	Gemma 4 12B and 31B / official QAT Q4_0
Software / inference hardware	Transformers / one T4	llama.cpp b11836 / two T4s, all model layers on GPUs
Turns / total output tokens	6 / 1,536	12 / 4,096
Output tokens per response / input limit	512 / 4,096	1,024 / 8,192
Time per attempt / generation	240 s / 80 s	240 s / 80 s
Attempts	24 cases × 3 methods = 72	24 cases × 3 methods × 2 models = 144

12B and 31B denote parameter counts. NF4 and Q4_0 are 4-bit weight formats; QAT means quantization-aware training. Tokens are model text units. Each case/model/method combination has one greedy run, which selects the highest-scoring next token, with seed 17 and the model's thinking mode disabled.

Study A's main comparison is ordinary tools against checklist; trace access is exploratory. One failed download produced no model attempts. Its permitted replacement supplies that batch's observations.

Study B reuses cases after Study A's results were inspected. Its settings were frozen separately before evaluation; it is not a new unseen test set. Both models share the corrected SQL permission checks, fixed model versions, schedule and limits. Each input has one verified beginning-of-sequence (BOS) token. The 31B stage runs first, followed by 12B. The main comparison is ordinary-tools recovery across models. Earlier startup and generation checks produced no benchmark attempts. We did not selectively rerun failed model attempts.

The two 12B studies use different software, weight formats, budgets and testing code. Their score difference cannot be assigned to one change. Within Study B, equal time limits affect models with different speeds. Checkpoint differences and fixed execution order also prevent attributing the observed advantage solely to parameter count.

4. Study A and the separate H08 correction

Method	**Full recovery**	**Visible recovery**	**Executable but wrong**
Ordinary tools	2/24	5/24	17/24
Checklist	1/24	7/24	19/24
Trace access	0/24	6/24	21/24

These original scores include a bug in the test harness, the code that runs the experiments. On H08, its SQLite permission checker rejects a valid read because SQLite reports the table without the expected database/column metadata. All three methods encounter this error. Checklist spends all six turns repeating the denied query. Those failures cannot be explained solely by the model's repair ability.

We fixed and versioned the permission checker, then reran all three H08 methods separately. Each achieved **0/1 full recovery**. Ordinary and checklist produced repairs that execute but leave historical data wrong. Trace access failed with a duplicate-key error. The incorrect permission denials disappeared. The original ordinary repair still passes with the corrected code; the rerun produced a different repair after receiving different feedback. We retain both sets of observations without replacing cells or combining scores.

All 60 original repairs that execute successfully are stable, including 57 wrong repairs. Of those 57, 42 already fail historical correctness on the visible data; 15 pass the visible checks but fail private variants. No trace calls occur. These results describe this bounded setup and retain the H08 caveat.

5. Study B: matched 12B and 31B

All 144 attempts complete. None is missing because of an infrastructure failure.

Method	12B full recovery	31B full recovery	31B minus 12B
Ordinary tools	11/24 (45.8%)	18/24 (75.0%)	+29.2 percentage points
Checklist	12/24 (50.0%)	17/24 (70.8%)	+20.8 points
Trace access	10/24 (41.7%)	16/24 (66.7%)	+25.0 points

With ordinary tools, nine cases succeed only with 31B, two only with 12B, nine with both and four with neither. Cases successful only with 31B versus only with 12B number eight versus three for checklist, and eight versus two for trace access. The larger model performs better overall, but not on every case.

With ordinary tools, 31B solves all four authorization cases and all four evidence-adjudication cases; 12B solves two and one respectively. For delivery cases, 12B solves four and 31B solves three. For ordered-state cases, 31B solves one and 12B solves none. These groups contain related authored cases, not independent application domains.

For 31B, checklist loses one ordinary-tools success and adds none; trace access loses two and adds none. For 12B, checklist adds two and loses one, while trace access adds one and loses two. The 31B model never calls trace, and 12B calls it once. We therefore cannot establish a consistent prompting benefit or judge a recovery strategy that actively uses provenance.

Each count below is out of 24 cases:

Model / method	Visible recovery	Executable but wrong
31B ordinary	21	4
31B checklist	21	6
31B trace access	22	8
12B ordinary	17	10
12B checklist	17	8
12B trace access	17	12

All 132 repairs that execute successfully are stable, including 48 wrong repairs. Stability rejects none of them. In both studies, every repair that passes historical correctness also passes all six checks. The extra checks reject selected incorrect controls, but they do not reject any additional historically correct model repair in these runs.

The full v3 job takes 127.68 minutes. Across its attempts, 31B uses 33,460 output tokens and 68.92 minutes; 12B uses 52,489 tokens and 49.80 minutes. Four 31B generation timeouts affect three attempts, one ending at its overall time limit. The 12B model hits a response-token limit twice and ends four attempts at the turn limit; one also exhausts total output tokens. These outcomes remain included.

6. Later CPU check and inspected failures

During manuscript review, we tested whether the supplied patches already contain a complete repair. One text rule removes filters of the form `WHERE key IN (SELECT entity FROM work)` and corresponding `AND` filters. It allows an optional table alias, removes two filters per case and preserves all other conditions. It uses no case-specific rules, reference repair SQL or private expected answers to construct repairs. We saved every candidate before grading it with the unchanged v3 checks.

CPU check, separate from model studies	Full recovery	Stable state
Unchanged patch used as recovery	0/24	24/24
No-op: leave the data unchanged	0/24	24/24
Supplied patch with work filters removed	**24/24**	24/24

Each score requires all three variants. This **post hoc check** was designed after inspecting tasks and model outcomes. It is not an advance-registered baseline or evidence on new tasks. Removing write filters can be unsafe under production constraints absent here.

This changes how failures should be read. On H17, 31B ordinary computes the intended meter value but limits recovery to literal `east_0`. It passes the visible data and fails renamed variants. On H12, its repair deletes a stale cache row but never restores the missing current row, so it fails even visibly. These are verified properties of the generated SQL, not explanations of the model's internal reasoning.

A no-op is also stable while leaving damage untouched. Stable but wrong behavior is therefore not a new property discovered by this work. In these runs, execution and stability do not separate correct from incorrect repairs. Displayed rows are evidence, not a correctness verdict.

7. Related work

WARP [1] and QFix [2] already address historical repair and later legitimate changes. Mahif replays modified histories while skipping irrelevant work [3]. Ultraverse combines application and database what-if execution [4]. Castle generates SQL updates with cascading effects [5]. Recovery-Bench tests agents starting from failed action sequences [6].

ArrivalBench tests generated pipelines under changing delivery schedules [7]; AfterFix starts with damaged projections and supplied SQL. DBA-Bench tests repairs and safety in active databases [8]; AfterFix uses small offline copies with complete evidence. AgentTether uses dependency graphs to guide recovery of agent action sequences [9]; our trace tool follows only supplied edges. Ye and Hu diagnose repairs by executing combinations of proposed and reference targets and updates [10]; our checks do not isolate causes that way. Our claim is limited to this focused task collection and evaluation.

8. Limitations and reproducibility

The shared work-queue design and 24/24 filter-removal result limit difficulty and diversity. Six groups do not remove this shortcut. We lack independent human domain review, human performance measurements and automatic recovery baselines on varied constructions. Similar instructions and rare trace use limit method conclusions.

A later internal AI-assisted review covered all 24 cases. Explicit initial and continuation answers matched the oracles in 144 comparisons across 72 case/seed combinations; 72 healthy-state checks and all admission checks passed. These are consistency checks, not independent validation or model observations.

Separate CPU probes exposed four boundaries. H04 and H12 lack rules for conflicting accepted results with equal sequence numbers; their SQL and oracles disagree on added tie inputs. H23 returns NULL in SQL but zero in the oracle for an added stream without partitions. H24 leaves a surviving child's parent pointer stale after parent erasure unless that child also enters `work`; the prompt does not specify this expansion. Evaluated fixtures contain neither ties nor empty streams, and their supplied work lists pass continuation checks. H19's wording also needs qualification: positive units applies to usage, while reset records have zero units. We disclose these gaps without rewriting frozen prompts or scores.

One run per combination cannot measure variability. Private variants retain case structure, and Study B reuses inspected cases. Complete evidence and static, rebuildable projections omit missing logs, external effects, concurrency and limited rebuilding windows. We make no population, significance or causal model-size claims. The original harness bug remains disclosed.

The versioned CPU package regrades all 219 saved model attempts, exactly replays 941 feedback records, rebuilds separate study tables and reproduces the scope diagnostic and internal review. It passes in an empty Python 3.12.14 environment with SQLite 3.53.1. This reproduces analysis from saved outputs, not model generation or GPU timings. The earlier v3 GPU audit verified 730 generation records. Original, corrected H08, v3, scope-diagnostic and review evidence remain separate. AI assistance supported case and code development, analysis, review and writing; the internal review was not independent human validation.

The local NF4 artifact v1 includes file checksums and a reproduction adapter. Public access and adapter GPU execution remain untested. These new hashes cannot prove historical weight bytes. The v3 QAT checkpoints are separate.

No funding was received. Copyright 2026 Nitish Songara. Original manuscript: CC BY 4.0 OR Apache 2.0, at the recipient's choice. Original code and synthetic benchmark: Apache 2.0. Third-party rights remain separate. Publication awaits approval.

References

1. Chandra et al. Intrusion recovery for database-backed web applications (WARP). SOSP, 2011.

2. Wang, Meliou and Wu. QFix: Diagnosing errors through query histories. SIGMOD, 2017.

3. Campbell, Arab and Glavic. Efficient Answering of Historical What-if Queries. 2022.

4. Ko et al. Ultraverse: A System-Centric Framework for Efficient What-If Analysis for Database-Intensive Web Applications. 2025.

5. Su et al. Castle: Causal Cascade Updates in Relational Databases with Large Language Models. EMNLP, 2025.

6. Letta. Introducing Recovery-Bench: Evaluating LLMs' Ability to Recover from Mistakes. 2025. Repository, accessed 24 October 2026.

7. Kothari. ArrivalBench: Agent-Generated Data Pipelines Are Correct Once and Wrong Under Time. 2026.

8. Chen et al. DBA-Bench: A Production-Fidelity Benchmark for LLM-Based Database Operations Agents. 2026.

9. Zhao et al. AgentTether: Graph-Guided Diagnosis and Runtime Intervention for Reliable LLM Agent Operation. 2026.

10. Ye and Hu. From Outcomes to Components: A Counterfactual Evaluation Framework for LLM-Proposed Repairs. 2026.